

Vòng lặp

Nguyễn Huyền Châu – AI Lab/TLU

Nội dung

- Các phép toán hữu ích
- Vòng lặp while
- Vòng lặp for
- Vòng lặp do/while
- Các lệnh break và continue
- Các ứng dụng của vòng lặp
- Mở rộng: Vòng lặp lồng
- Mở rộng: Làm việc với file

Các phép toán hữu ích

- ❑ Phép gán kết hợp
- ❑ Phép tăng, giảm
- ❑ Dạng tiền tố, hậu tố

Phép gán kết hợp

Toán tử	Ví dụ	Ý nghĩa
<code>+=</code>	<code>x += 5;</code>	<code>x = x + 5;</code>
<code>-=</code>	<code>y -= 2;</code>	<code>y = y - 2;</code>
<code>*=</code>	<code>z += 10;</code>	<code>z = z + 10;</code>
<code>/=</code>	<code>a /= b;</code>	<code>a = a / b;</code>
<code>%=</code>	<code>c %= 3;</code>	<code>c = c % 3;</code>

- Sau khi phép tính thực hiện xong, biểu thức gán kết hợp sẽ mang kiểu và giá trị bằng với toán hạng trái.

```
x =2;
```

```
cout << (x += 5) << endl;
```

```
// giá trị của biểu thức x += 5 cũng bằng giá trị của biến x, tức là 7
```

Phép tăng, giảm, dạng tiền tố, hậu tố

- Toán tử tăng (**++**): **++c** làm c tăng 1
- Toán tử giảm (**--**): **--c** làm c giảm 1
- Dạng tiền tố:
 - **++c** hoặc **--c**
 - thực hiện phép tính trước, trả về giá trị sau
 - biểu thức chứa **++c** sẽ tính theo giá trị c mới sau khi tăng/giảm
- Dạng hậu tố:
 - **c++** hoặc **c--**
 - trả về giá trị trước, thực hiện phép tính sau
 - biểu thức chứa **++c** sẽ tính theo giá trị c cũ trước khi tăng/giảm
- Dùng với lvalue (vd: biến), không dùng được cho hằng và biểu thức:
++(num1 + num2) // báo lỗi

Dạng tiền tố vs. hậu tố

- Ví dụ về khác biệt giữa dạng tiền tố và hậu tố:

```
int c = 5;  
cout << ++c << endl;    // tăng c lên 6, rồi in ra c là 6
```

```
int c = 5;  
cout << c++ << endl;    // in c là 5, rồi tăng c lên 6
```

```
int c = 5;  
if (c++ == 5) cout << "c++: " << c << endl; // kiểm tra c == 5 rồi tăng c, sau đó in ra c, lúc này là 6  
int c = 5;  
if (++c == 5) cout << "++c: " << c << endl; // tăng c lên 6, rồi kiểm tra c == 5, vì false nên không in ra
```

- Nếu giá trị biểu thức không được sử dụng, tăng trước hay tăng sau là như nhau

c++;	++c;
cout << c;	cout << c;

là như nhau

Vòng lặp while

- ❑ Cấu trúc lặp while
- ❑ Điều khiển lặp bằng con đếm
- ❑ Điều khiển lặp bằng lính canh
- ❑ Cách thiết kế thuật toán kiểu top-down

Cấu trúc lặp while

- Giúp hành động được lặp đi lặp lại khi một điều kiện vẫn đúng
`while` (<điều kiện>) *giống if, điều kiện luôn để trong ()*
 <câu lệnh>

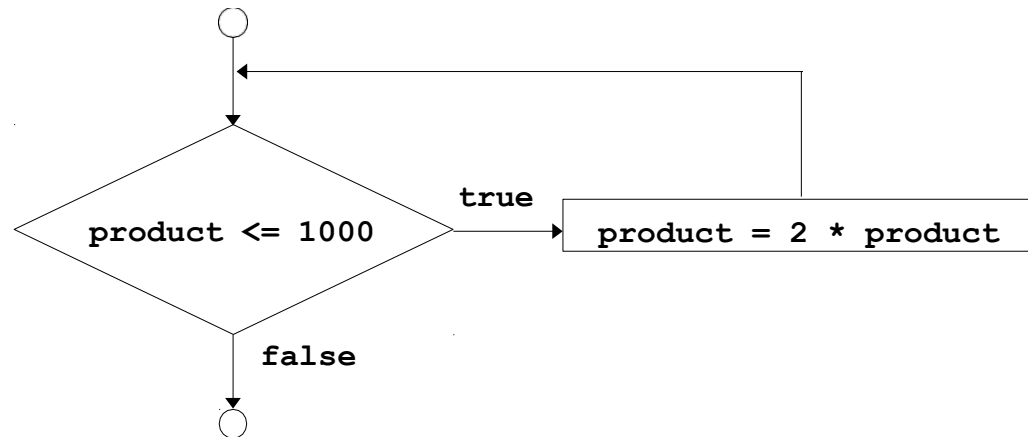
Trong khi vẫn còn tên hàng trong danh sách đi chợ

Mua mặt hàng tiếp theo và gạch tên nó ra khỏi danh sách

- Ngay khi điều kiện trở thành sai, while sẽ dừng chạy <câu lệnh>
- Ví dụ:

```
int product = 2;  
while ( product <= 1000 )  
|   product = 2 * product;
```

- dừng khi product = ??



Điều khiển lặp bằng con đếm

- Khi cần lặp một số lần xác định trước, có thể dùng con đếm (counter)
 - Là một biến kiểu nguyên để đếm số bước đã lặp
 - Trước vòng lặp, khởi tạo counter bằng 0
 - Điều kiện lặp là $(\text{counter} < n)$ với n là số lần lặp
 - Nhớ tăng counter thêm 1 trong thân vòng lặp

- *Nhập điểm thi của 10 sinh viên, tính trung bình điểm thi của lớp.*

```
int c = 0; // tên là c, count, counter
int tong = 0;
while (c < 10){
    cout << "Sinh vien " << c << ": ";
    int diem;
    cin >> diem;
    tong += diem;
    c++;
}
cout << "Trung binh: " << float(tong)/10 << endl;
```

- Vì sao điều kiện là $c < n$ mà không phải $c \leq n$??
- Nếu khởi tạo $c = 1$??
- Sau vòng lặp, $c =$??
- Nếu bỏ dòng $c++$??

- Con đếm cũng có thể chạy trong $[a,b]$: khởi tạo??

Thực hành: Điều khiển lặp bằng con đếm

// Viết chương trình yêu cầu người dùng nhập một giá trị nguyên dương. Chương trình nên
// sử dụng một vòng lặp để lấy tổng của tất cả các số nguyên từ 1 đến số được nhập. VD:
// Nếu người dùng nhập 50, vòng lặp sẽ tìm tổng của 1, 2, 3, 4 ,. . . , 50.

```
#include <iostream>
using namespace std;
int main()
{
    int n;
    cout << "n: ";
    cin >> n;

    int c = 1, tong = 0;
    while (c <= n){
        tong += c;
        c++;
    }
    cout << "Tổng các số từ 1 đến " << n << " là: " << tong << endl;

    return 0;
}
```

Con đếm chạy trong khoảng [1, n]

n: 50
Tổng: 1275

Điều khiển lặp bằng lính canh

- Giả sử bài toán tính điểm trở thành:
 - *Viết chương trình tính điểm trung bình của lớp, chương trình sẽ xử lý một số lượng điểm tùy ý mỗi khi chạy chương trình.*
 - Số sinh viên không biết trước, làm sao để dừng vòng lặp?
- Giá trị canh:
 - Là một giá trị đặc biệt báo hiệu dừng nhập liệu
 - Vòng lặp kết thúc khi phát hiện canh vừa được nhập
 - Canh được chọn để không lẫn với dữ liệu thông thường
 - Vd: nếu nhập điểm, chọn canh là -1 (hay số âm nào đó).

Thiết kế thuật toán

- Nhiều chương trình thường có 3 pha:
 - Khởi tạo - Initialization
 - *Khởi tạo các biến chương trình*
 - Xử lý - Processing
 - *Nhập liệu, điều chỉnh các biến trong chương trình*
 - Kết thúc - Termination
 - *Tính kết quả cuối và in ra*
 - Giúp dễ chia nhỏ chương trình và làm mịn từ trên xuống
- Ta sẽ tập thiết kế chương trình theo cấu trúc trên.

Thiết kế thuật toán

- Thiết kế từ trên xuống, làm mịn dần:
 - Bắt đầu bằng mã giả cho mức cao nhất
Tính trung bình điểm thi của lớp
 - Chia thành các nhiệm vụ nhỏ hơn, liệt kê theo thứ tự:
Khởi tạo các biến
Nhập, tính tổng, và đếm các điểm thi
Tính và in trung bình điểm thi

Thiết kế thuật toán

- Làm mịn pha khởi tạo:

Khởi tạo các biến

thành

Khởi tạo tổng bằng 0

Khởi tạo biến đếm bằng 0

- Pha xử lý:

Nhập, tính tổng, và đếm các điểm thi

thành

Nhập điểm đầu tiên (có thể là canh)

Trong khi người dùng còn chưa nhập canh

Cộng điểm vừa nhập vào tổng

Cộng thêm 1 vào biến đếm điểm

Nhập điểm tiếp theo (có thể là canh)

Thiết kế thuật toán

- Pha kết thúc:

Tính và in trung bình điểm thi
thành

Nếu con đếm khác 0

Đặt trung bình bằng tổng chia cho con đếm

In giá trị trung bình

Nếu không

In “Không nhập điểm nào”

- Bước tiếp theo: mã C++

Ví dụ: Điều khiển lặp bằng canh

// Viết chương trình tính điểm trung bình của lớp, chương trình sẽ xử lý
// một số lượng điểm tùy ý mỗi khi chạy chương trình.

```
#include <iostream>
using namespace std;
int main()
{
    double tong = 0;
    int dem = 0;
    double diem;
    cout << "Diem: ";
    cin >> diem;
    while (diem != -1){
        dem++;
        tong += diem;
        cout << "Diem: ";
        cin >> diem;
    }
    if (dem)
        cout << "Trung binh: " << tong/dem << endl;
    else
        cout << "Khong co \n";
    return 0;
}
```

Nếu không là canh thì mới xử lý và nhập tiếp, nhập xong quay về đầu vòng lặp và tiếp tục kiểm tra có phải canh

Diem: 5
Diem: 7
Diem: 8
Diem: 10
Diem: -1
Trung binh: 7.5

Diem: -1
Khong co

Thực hành: Điều khiển lặp bằng cạnh

// Viết chương trình yêu cầu người dùng nhập vào các số nguyên, kết thúc khi nhập -1.
// Chương trình cần tính tổng của tất cả các số nguyên nhập vào trừ số -1. Ví dụ,
// nếu người dùng nhập 1, -5, 7, 2, -1 thì in ra tổng các số là $1 + (-5) + 7 + 2 = 5$

```
#include <iostream>
using namespace std;
int main()
{
    int tong = 0;
    double so;
    cout << "Nhập so (khac -1):";
    cin >> so;
    while (so != -1) {
        tong += so;
        cout << "Nhập so (khac -1):";
        cin >> so;
    }
    cout << "Tong: " << tong << endl;
    return 0;
}
```

```
Nhap so (khac -1): 1
Nhap so (khac -1): -5
Nhap so (khac -1): 7
Nhap so (khac -1): 2
Nhap so (khac -1): -1
Tong: 5
```

```
Nhap so (khac -1): -1
Tong: 0
```

Lưu ý với cấu trúc lặp while

- while là vòng lặp kiểm tra trước:

```
int number = 6;
while (number <= 5)
{
    cout << "Hello\n";
    number++;
}
```

Điều kiện được kiểm tra trước khi bắt đầu while

- Nếu sai ngay từ đầu thì vòng lặp không chạy lần nào

- Cần thận vòng lặp vô hạn (infinite loop):

```
int number = 1;
while (n<=5)
{
    cout << "Hello\n";
}
```

```
int number = 1;
while (1)
{
    cout << "Hello\n";
    number++;
}
```

- Vòng lặp phải chứa mã làm điều kiện sẽ thành sai
- Hoặc, chứa một lệnh thoát vòng lặp (**break**, **return**, **exit** ~ học sau)

Thực hành: Cấu trúc lặp while

- VD1: Tính tổng các số nguyên từ 1 đến n, n nhập từ bàn phím
- VD2: Vào sinh nhật, Bông được tặng n gói kẹo, mỗi gói lại chứa một số lượng kẹo nào đó. Nhập từ bàn phím 2 dòng như sau:
 - Dòng đầu chứa số n
 - Dòng thứ 2 chứa n số cách bởi dấu cách, là số kẹo trong n gói
 - Với đầu vào trên, viết chương trình tính số kẹo của bạn Bông

<u>Đầu vào</u>	<u>Đầu ra</u>
4 2 4 3 8	17

- VD3: “Vừa gà vừa chó, bó lại cho tròn, ba mươi sáu con, một trăm chân chẵn”. Từ bài toán dân gian này, mở rộng thành chương trình nhập vào tổng số gà và chó cùng tổng số chân, rồi in ra số gà, số chó.

<u>Đầu vào</u>	<u>Đầu ra</u>
36 100	chicken = 22, dog = 14

Vòng lặp for

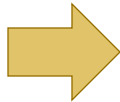
Ví dụ: Từ while đến for

// Viết chương trình yêu cầu người dùng nhập một giá trị nguyên dương. Chương trình nên
// sử dụng một vòng lặp để lấy tổng của tất cả các số nguyên từ 1 đến số được nhập. VD:
// Nếu người dùng nhập 50, vòng lặp sẽ tìm tổng của 1, 2, 3, 4 ,. . . , 50.

```
int main()
{
    int n;
    cout << "n: ";
    cin >> n;

    int c = 1, tong = 0;
    while (c <= n){
        tong += c;
        c++;
    }
    cout << "Tong: " << tong << endl;

    return 0;
}
```



```
int main()
{
    int n;
    cout << "n: ";
    cin >> n;

    int tong = 0;
    for (int i = 1; i <= n; i++)
        tong += i;
    cout << "Tong: " << tong << endl;

    return 0;
}
```

Cấu trúc lặp for

- Cú pháp

```
for (<khởi tạo> ; <điều kiện> ; <tăng/giảm con đếm>)  
    <câu lệnh>
```

- Tương đương với:

```
<khởi tạo> ;  
while (<điều kiện>){  
    <câu lệnh>  
    <tăng/giảm con đếm>;  
}
```

- for cũng là vòng lặp kiểm tra trước:

```
for (count = 11; count <= 10; count++) // không bao giờ chạy  
    cout << "Hello" << endl;
```

Lưu ý với cấu trúc lặp for

- Có thể kết hợp cả khai báo trong phần khởi tạo

```
for (int i = 0; i < 10; i++) // i chỉ tồn tại trong for, hết for sẽ bị huỷ
```

- Tiết kiệm bộ nhớ + dùng chung tên biến cho nhiều vòng for

- Nếu dùng nhiều biến đếm hay nhiều lệnh tăng giảm, dùng dấu phẩy để tách

```
for (int i = 0, j = 0; j + i <= 10; j++, i++)  
    cout << j + i << endl;
```

- Có thể để trống bất kỳ phần nào nếu muốn:

```
int i = 0;  
for (; i < 10; i++) // khởi tạo i trước for nên không cần làm tiếp  
  
for (;;) {  
    //...           Vòng for ever  
}
```

Thực hành: Vòng for

// Mức độ của đại dương:

// Giả sử mực nước biển đang tăng khoảng 1,5 milimet mỗi năm, hãy viết chương trình

// hiển thị số milimet mà đại dương sẽ tăng lên mỗi năm trong 25 năm tới.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    const double tang = 1.5;
```

```
    for (int i = 1; i <= 25; i++)
```

```
    {
```

```
        cout << "Muc nuoc bien nam thu " << i << " tang: " << i * tang << " mm. \n";
```

```
    }
```

```
Muc nuoc tang trong nam 1: 1.5mm.
Muc nuoc tang trong nam 2: 3mm.
Muc nuoc tang trong nam 3: 4.5mm.
Muc nuoc tang trong nam 4: 6mm.
Muc nuoc tang trong nam 5: 7.5mm.
Muc nuoc tang trong nam 6: 9mm.
Muc nuoc tang trong nam 7: 10.5mm.
Muc nuoc tang trong nam 8: 12mm.
Muc nuoc tang trong nam 9: 13.5mm.
Muc nuoc tang trong nam 10: 15mm.
Muc nuoc tang trong nam 11: 16.5mm.
Muc nuoc tang trong nam 12: 18mm.
Muc nuoc tang trong nam 13: 19.5mm.
Muc nuoc tang trong nam 14: 21mm.
Muc nuoc tang trong nam 15: 22.5mm.
Muc nuoc tang trong nam 16: 24mm.
Muc nuoc tang trong nam 17: 25.5mm.
Muc nuoc tang trong nam 18: 27mm.
Muc nuoc tang trong nam 19: 28.5mm.
Muc nuoc tang trong nam 20: 30mm.
Muc nuoc tang trong nam 21: 31.5mm.
Muc nuoc tang trong nam 22: 33mm.
Muc nuoc tang trong nam 23: 34.5mm.
Muc nuoc tang trong nam 24: 36mm.
Muc nuoc tang trong nam 25: 37.5mm.
```

Biến `i` được khai báo lần khởi tạo bên trong `for`, nên là biến cục bộ chỉ tồn tại trong `for`, sẽ bị huỷ khi `for` kết thúc

Thực hành: Vòng for

// Tính ca-lo:
// Chạy trên một máy chạy bộ sẽ đốt cháy 3,6 calo mỗi phút. Viết chương trình
// hiển thị số calo đã đốt cháy sau 5, 10, 15, 20, 25 và 30 phút.

```
#include <iostream>
using namespace std;
int main()
{
    const float motPhut = 3.6;

    //hien thi so ca-lo theo cac moc thoi gian
    for (int i = 5; i <= 30; i += 5)
        cout << i << " : " << i * motPhut << " calories " << endl;
    return 0;
}
```

Có mốc bắt đầu, mốc kết thúc, có bước nhảy
là viết được for

```
5 : 18 calories
10 : 36 calories
15 : 54 calories
20 : 72 calories
25 : 90 calories
30 : 108 calories
```

Vòng lặp do/while

Cấu trúc lặp do/while

■ Tương tự while

- kiểm tra điều kiện ở cuối thay vì đầu
do

<câu lệnh>

while (<điều kiện>) ; *← Lưu ý sau điều kiện có ;*

- lần chạy đầu tiên không kiểm tra
- => chạy ít nhất 1 lần, bất kể điều kiện

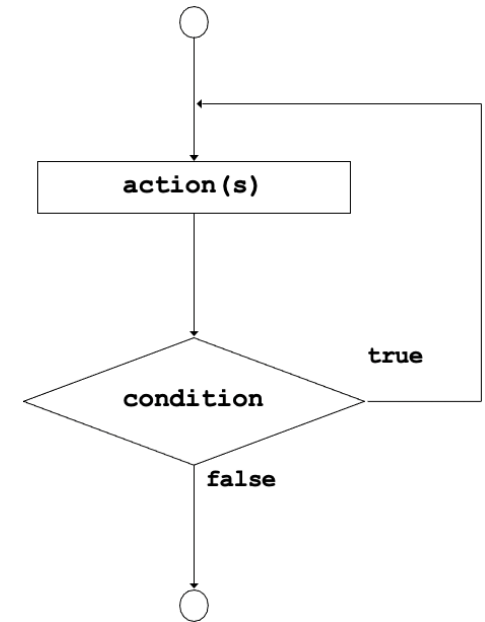
do

```
|    cout << "Hello\n";
```

```
|    while (0);
```

Hello

- điều kiện (0) luôn nghĩa là **false**, nhưng lệnh in vẫn sẽ chạy được 1 lần



Ví dụ: Vòng do/while

```
// yêu cầu người dùng nhập một xâu rồi in lại xâu đó  
// tiếp tục làm vậy cho đến khi người dùng nhập "good bye"
```

```
#include <iostream>  
using namespace std;  
int main(){  
    string s;  
    do {  
        cout << "Nhập xâu: ";  
        getline(cin, s);  
        cout << "Bạn vừa nhập: " << s << endl;  
    }while (s != "good bye");  
    return 0;  
}
```

Nhập xâu: when you say nothing at all
Bạn vừa nhập: when you say nothing at all
Nhập xâu: save the best for last
Bạn vừa nhập: save the best for last
Nhập xâu: good bye
Bạn vừa nhập: good bye

Nhập xâu: good bye
Bạn vừa nhập: good bye

Còn chưa gặp "good bye" thì còn làm

Ví dụ: Bài toán xác minh đầu vào

// yêu cầu nhập một số nguyên chẵn, nếu người dùng nhập số lẻ thì lại mời nhập
// cho đến nào chẵn thì dừng, cuối cùng in ra số nguyên chẵn này

```
#include <iostream>
using namespace std;
int main(){
    int n;
    do {
        cout << "Nhập số chẵn: ";
        cin >> n;
    }while (n % 2);
    cout << "Số chẵn: " << n << endl;
    return 0;
}
```

Viết tắt cho $(n \% 2 \neq 0)$

```
Nhap so chan: 3
Nhap so chan: 5
Nhap so chan: 1
Nhap so chan: 6
So chan: 6
```

Bài toán xác minh đầu vào

- Xác minh đầu vào là quá trình kiểm tra xem dữ liệu nhập vào có hợp lệ.
- Vì kiểm tra chỉ xảy ra sau khi đã nhập liệu, ta sẽ dùng do/while là vòng kiểm tra sau:

```
do{  
    mời nhập  
    nhập liệu  
}while (dữ liệu còn sai);
```

```
int n; // cần n <=10;  
do{  
    cout << "n: ";  
    cin >> n;  
}while (n>10);
```

Thực hành: Xác minh đầu vào dùng do/while

```
#include <iostream>
using namespace std;
int main()
{
    double speed;
    do
    {
        cout << "Nhap toc do (>= 0): ";
        cin >> speed;
    }
    while (speed < 0); // tốc độ âm thì nhập lại

    int time;
    do
    {
        cout << "Nhap so gio (>= 1): ";
        cin >> time;
    }
    while(time < 1); // số giờ < 1 thì nhập lại

    for (int i = 1; i <= time; i++)
        cout << "Quang duong di duoc sau " << i << " gio la " << i * speed << " km. \n ";
    return 0;
}
```

// Nhập vào tốc độ của một chiếc xe và số giờ
// đã đi. Sau đó, hiển thị quãng đường xe đã đi
// trong mỗi giờ của khoảng thời gian đó.
// Không chấp nhận tốc độ < 0 và số giờ < 1

Toc do xe: 40

So gio da di: 3

Khoang cach theo gio ma xe da di:

1	40
2	80
3	120

Nhap toc do (>= 0): -3

Nhap toc do (>= 0): -4

Nhap toc do (>= 0): -2

Nhap toc do (>= 0): 40

Nhap so gio (>= 1): -3

Nhap so gio (>= 1): -2

Nhap so gio (>= 1): 0

Nhap so gio (>= 1): 3

Quang duong di duoc sau 1 gio la 40 km.

Quang duong di duoc sau 2 gio la 80 km.

Quang duong di duoc sau 3 gio la 120 km.

Thực hành: Xác minh đầu vào dùng do/while

// Lượng mưa trung bình:
// Hỏi người dùng lượng mưa trong mỗi tháng trong năm. Sau đó in ra
// lượng mưa trung bình. Không chấp nhận lượng mưa hàng tháng âm.

```
#include <iostream>
using namespace std;
int main()
{
    double tongMua = 0;
    cout << "Nhap luong mua hang thang: \n";
    for (int i = 1; i <= 12; i++)
    {
        double mua;
        do {
            cout << i << " : ";
            cin >> mua;
        } while (mua < 0);
        tongMua += mua;
    }
    cout << "Luong mua trung binh hang thang: " << tongMua/12 << endl;
    return 0;
}
```

Còn âm thì còn nhập

Thoát while thì hết âm, được phép cộng gộp vào tổng lượng mưa

Bài toán lặp lại menu

- Một menu ít khi hiển thị chỉ một lần, mà thường hiển thị liên tục cho đến khi người dùng không muốn chọn nữa.
 - tự nhiên cần dùng vòng lặp
- Vì hiển thị menu **ít nhất một lần**, nên ta dùng vòng do/while

khởi tạo biến lựa chọn

do{

in ra menu và lời mời nhập lựa chọn

đọc lựa chọn

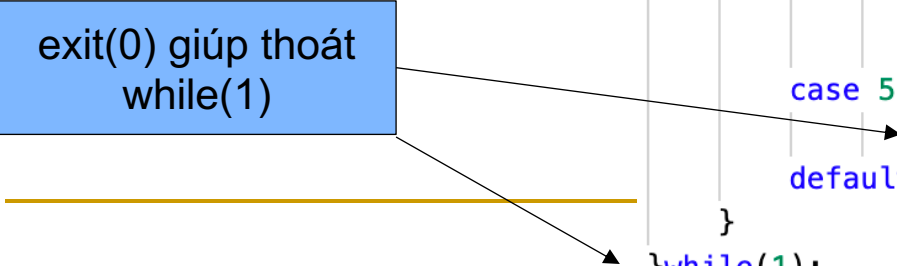
xét lựa chọn để thực hiện yêu cầu người dùng

} while (1) ← Vì sao có thể dùng vòng vô hạn??

Ví dụ: Lặp lại menu với do/while

// Viết chương trình hiển thị 2 số ngẫu nhiên, sau đó hiển thị thực đơn lựa chọn các phép toán:
// cộng, trừ, nhân, chia và chức năng thoát. Sau khi thực hiện phép toán, chương trình sẽ hiển
// thị lại thực đơn để người dùng có thể lựa chọn phép toán khác. Quá trình này được lặp lại
// cho đến khi người dùng chọn thoát

```
int main(){  
    int a, b;  
    srand(time(0));  
    a = rand() % 10000 + 1 ;  
    b = rand() % 10000 + 1 ;  
    cout << a << " " << b << endl;  
  
    do{  
        int c;  
        cout << "Ban muon lam gi:\n"  
        << "1. Tinh tong \n"  
        << "2. Tinh hieu \n"  
        << "3. Tinh tich \n"  
        << "4. Tinh thuong \n"  
        << "5. Thoat \n";  
        cin >> c;  
  
        switch (c){  
            case 1: cout << "Tong hai so: " << a + b << endl; break;  
            case 2: cout << "Hieu hai so: " << a - b << endl; break;  
            case 3: cout << "Tich hai so: " << a * b << endl; break;  
            case 4: if (!b) cout << "Mau so bang 0, khong the chia\n";  
                    else cout << "Thuong hai so: " << a + b << endl;  
                    break;  
            case 5: cout << "Thoat chuong trinh ... " << endl;  
                    exit(0);  
            default: cout << "Lua chon khong ton tai " << endl;  
        }  
    }while(1);  
    return 0;  
}
```



exit(0) giúp thoát while(1)

Thực hành: Lặp lại menu với do/while

```
int main()
{
    do{
        int choice;
        cout << "Ban muon tinh dien tich cua hinh nao? \n";
        << "1. Hinh tron \n"
        << "2. Hinh chu nhat \n"
        << "3. Hinh tam giac vuong \n"
        << "4. Thoat \n" ;
        cin >> choice;
        float r, dai, rong, a, b;
        const float PI = 3.14;
        switch (choice){
            case 1: cout << "Ban kinh: ";
                    cin >> r;
                    cout << "Dien tich hinh tron: " << PI * r * r << endl;
                    break;
            case 2: cout << "Chieu dai, chieu rong: ";
                    cin >> dai >> rong;
                    cout << "Dien tich hinh chu nhat: " << dai * rong << endl;
                    break;
            case 3: cout << "Hai canh goc vuong: ";
                    cout << "Dien tich hinh tam giac vuong: " << a * b/2 << endl;
                    break;
            case 4: cout << "Thoat ... \n";
                    exit(0);
            default: cout << "Loi! Lua chon nay khong co trong danh sach! \n";
        }
    }while(1);
    return 0;
}
```

Lệnh break và continue

- ❑ Câu lệnh break
- ❑ Câu lệnh continue

Lệnh break

- Giúp thoát khỏi các cấu trúc while, for, do/while, switch
- Chương trình tiếp tục chạy tại lệnh đầu tiên ngay sau cấu trúc
- Thường được dùng để:
 - ❑ Thoát khỏi vòng lặp sớm hơn bình thường
 - ❑ Bỏ qua phần còn lại của switch

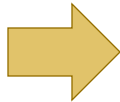
```
for (int i = 0; i<10; i++){  
    if (i==5) break;  
    cout << i * i << " ";  
}
```

0 1 4 9 16

Thực hành: Lệnh break

// Viết chương trình tính điểm trung bình của lớp, chương trình sẽ xử lý
// một số lượng điểm tùy ý mỗi khi chạy chương trình.

```
int main()
{
    double tong = 0;
    int dem = 0;
    double diem;
    cout << "Diem: ";
    cin >> diem;
    while (diem != -1){
        dem++;
        tong += diem;
        cout << "Diem: ";
        cin >> diem;
    }
    if (dem)
        cout << "Trung binh: " << tong/dem << endl;
    else
        cout << "Khong co \n";
    return 0;
}
```



```
int main()
{
    double tong = 0;
    int dem = 0;
    while (1){
        double diem;
        cout << "Diem: ";
        cin >> diem;
        if (diem == -1) break;
        dem++;
        tong += diem;
    }
    if (dem)
        cout << "Trung binh: " << tong/dem << endl;
    else
        cout << "Khong co \n";
    return 0;
}
```

Xử lý dữ liệu kết thúc bởi canh

- Dữ liệu phải khác giá trị canh mới được xử lý, còn là canh thì kết thúc nhập liệu:

while (1){

Mời nhập

Nhập liệu

if (dữ liệu bằng canh)

Thoát

Xử lý

}

```
while (1){  
    double diem;  
    cout << "Diem: ";  
    cin >> diem;  
    if (diem == -1) break;  
    dem++;  
    tong += diem;  
}
```

Thực hành: Xử lý dữ liệu kết bởi cạnh

// Viết chương trình yêu cầu người dùng nhập vào các số nguyên, kết thúc khi nhập -1.
// Chương trình cần tính tổng của tất cả các số nguyên nhập vào trừ số -1. Ví dụ,
// nếu người dùng nhập 1, -5, 7, 2, -1 thì in ra tổng các số là $1 + (-5) + 7 + 2 = 5$

```
#include <iostream>
using namespace std;
int main()
{
    int tong = 0;
    while (1) {
        double so;
        cout << "Nhap so (khac -1):";
        cin >> so;
        if (so == -1) break;
        tong += so;
    }
    cout << "Tong: " << tong << endl;
    return 0;
}
```

Ba đoạn mã đều dùng được

```
do {
    double so;
    cout << "Nhap so (khac -1):";
    cin >> so;
    if (so == -1) break;
    tong += so;
}while (1);

for (;;) {
    double so;
    cout << "Nhap so (khac -1):";
    cin >> so;
    if (so == -1) break;
    tong += so;
}
```


Thực hành: Xử lý dữ liệu kết bởi cạnh

// Dữ liệu kết bởi cạnh là số chẵn:

// Viết chương trình nhập vào từng số nguyên rồi in ra luôn số đó, dừng khi nhập vào số chẵn.

```
#include <iostream>
using namespace std;
int main(){
    while(1) {
        cout << "n: ";
        cin >> n;
        if (n % 2 == 0) break;
        cout << "Số vua nhap: " << n;
    };
    return 0;
}
```

Lệnh continue

- Được dùng cho while, for, do/while
 - ❑ Bỏ qua phần còn lại của thân vòng lặp
 - ❑ Chạy tiếp lần lặp kế nếu còn
- Với while và do/while:
 - ❑ Kiểm tra điều kiện lặp ngay sau continue
- Với for:
 - ❑ Phần <tăng/giảm biến đếm> được thực hiện
 - ❑ Rồi điều kiện lặp được kiểm tra

```
for (int i = 0; i<10; i++){  
    if (i==5) continue;  
    cout << i*i << " ";  
}
```

0 1 4 9 16 36 49 64 81

Thực hành: Lệnh continue

// Nhập vào hai số nguyên m và n, giả định m nhỏ hơn n. In ra bình phương, căn bậc 2, loga cơ số e, lũy thừa cơ số e của mọi số từ 1 đến n, trừ với số m.

```
#include <iostream>
#include <cmath>
#include <iomanip>
using namespace std;
int main()
{
    int m, n;
    cout << "m, n : ";
    cin >> m >> n;
    cout << setprecision (2) << fixed;
    for (int i = 1; i<=n; i++){
        if (i == n) continue;
        cout << i << " : " << i * i << "\t" << sqrt(i) << "\t"
            << log(i) << "\t" << exp(i) << endl;
    }
    return 0;
}
```

m, n : 3 7			
1 : 1	1.00	0.00	2.72
2 : 4	1.41	0.69	7.39
3 : 9	1.73	1.10	20.09
4 : 16	2.00	1.39	54.60
5 : 25	2.24	1.61	148.41
6 : 36	2.45	1.79	403.43

Nhảy qua phần còn lại của lượt lặp, đến ngay kiểm tra điều kiện (i<=n)

Các ứng dụng của vòng lặp

- ❑ Cách lựa chọn vòng lặp
- ❑ Tính toán với biến tích lũy

Ví dụ: while vs. for vs. do/while

// Mức độ của đại dương:

// Giả sử mực nước biển đang tăng khoảng 1,5 milimet mỗi năm, hãy viết chương trình

// hiển thị số milimet mà đại dương sẽ tăng lên mỗi năm trong 25 năm tới.

```
#include <iostream>
using namespace std;
int main()
{
    const double tang = 1.5;
    for (int i = 1; i <= 25; i++)
        cout << "Muc nuoc bien nam thu " << i << " tang: " << i * tang << " mm. \n";
    return 0;
    i++;
}while (i <= 25);
return 0;
}
```

Lựa chọn vòng lặp

- Mọi vòng lặp đều viết được tương đương sang vòng lặp khác song vẫn có ưu nhược khác nhau nên vẫn cần sự lựa chọn
- **for** là vòng kiểm tra trước
 - Có sẵn phần khởi tạo, điều kiện, tăng giảm
 - Hay dùng khi biết trước số lần lặp
- **while** là vòng lặp kiểm tra trước
 - Hay dùng để xử lý dữ liệu kết thúc bởi canh (kết hợp với break)
 - Và khi không biết số lần lặp
- **do/while** là vòng lặp kiểm tra sau
 - Hay dùng khi xác minh đầu vào; lặp lại menu
 - Và khi không biết số lần lặp nhưng biết lặp ít nhất một lần

Tính toán với biến tích lũy

- Biến tích lũy: lưu thông tin tích lũy khi duyệt qua nhiều phần tử
 - vd đếm, tính tổng, tìm nhỏ nhất, tìm khoá, ...
- Biến tích lũy
 - khai báo trước vòng lặp (dem, tong, min,...)
 - khởi tạo phù hợp: dem = ? tong = ?, tích = ?
 - cần cập nhật qua từng bước duyệt phần tử
- Dùng vòng for nếu đã biết số phần tử, dùng while nếu chưa biết

Ví dụ: Cộng tích lũy

// Viết chương trình yêu cầu người dùng nhập một giá trị nguyên dương. Chương trình nên
// sử dụng một vòng lặp để lấy tổng của tất cả các số nguyên từ 1 đến số được nhập. VD:
// Nếu người dùng nhập 50, vòng lặp sẽ tìm tổng của 1, 2, 3, 4 ,. . . , 50.

```
#include <iostream>
using namespace std;
int main()
{
    int n;
    cout << "n: ";
    cin >> n;

    int tong = 0;
    for (int i = 1; i <= n; i++)
        tong += i;
    cout << "Tong: " << tong << endl;

    return 0;
}
```

n: 50
Tong: 1275

Thực hành: Nhân tích lũy

// Tăng phí thành viên:

// Một câu lạc thu phí thành viên mỗi năm 2.500 đô la và sẽ tăng phí thêm 4% mỗi năm
// trong 6 năm tới. Viết chương trình hiển thị mức phí dự kiến trong 6 năm tới.

```
#include <iostream>
#include <iomanip>
using namespace std;
```

```
int main()
{
    double const tang = 1 + 0.04;
    double phi = 2500;

    cout << fixed << showpoint << setprecision(2);
    for (int i = 1; i <= 6; i++)
    {
        phi *= tang;
        cout << "Nam " << i << " : " << phi << " $ \n";
    }
    return 0;
}
```

Nam 1	:	2600.00	\$
Nam 2	:	2704.00	\$
Nam 3	:	2812.16	\$
Nam 4	:	2924.65	\$
Nam 5	:	3041.63	\$
Nam 6	:	3163.30	\$

Tăng 4% của năm trước đó (vốn đã tăng so với năm đầu), không phải tăng 4% của năm đầu => tích lũy kiểu nhân, không phải kiểu cộng

Thực hành: Tìm min, max

// Lớn nhất và nhỏ nhất trong các số:

// Nhập một dãy các số nguyên, kết bởi -1. Sau đó in ra số nhỏ nhất và lớn nhất đã nhập.

```
int main()
{
    int min, max;
    int n;
    cout << "Nhập số (khác -1): ";
    cin >> n;

    if (n != -1){
        min = max = n;
        while (1){
            cout << "Nhập số (khác -1): ";
            cin >> n;
            if (n == -1) break;
            if (n < min ) min  = n;
            if (n > max) max = n;
        }
        cout << "Nhỏ nhất: " << min << endl
              << "Lớn nhất: " << max << endl;
    }
    else cout << "Không nhập vào số nào \n";

    return 0;
}
```

```
Nhập số (khác -1): 4
Nhập số (khác -1): -2
Nhập số (khác -1): 5
Nhập số (khác -1): 7
Nhập số (khác -1): -3
Nhập số (khác -1): 21
Nhập số (khác -1): 8
Nhập số (khác -1): -1
Nhỏ nhất: -3
Lớn nhất: 21
```

```
Nhập số (khác -1): -1
Không nhập vào số nào
```

Mở rộng: Vòng lặp lồng

Vòng lặp lồng

- Là vòng lặp nằm trong vòng lặp khác:

```
for (row=1; row<=3; row++)    // vòng ngoài
    for (col=1; col<=3; col++) // vòng trong
        cout << row * col << endl;
```

- Mỗi bước vòng ngoài sẽ gọi **mọi bước** vòng trong. Vòng trong sẽ bắt đầu, lặp mọi bước, kết thúc, rồi lại bắt đầu, lặp mọi bước, kết thúc, ...
- Tổng lần lặp của vòng trong bằng tích số bước lặp của 2 vòng.

Ví dụ: Vòng lặp lồng for

// Nhập vào 2 số nguyên là số sinh viên và số bài kiểm tra mỗi sinh viên đều phải làm.
// Sau đó chạy vòng lặp nhập điểm từng bài kiểm tra cho từng sinh viên, rồi hiển thị điểm
// trung bình kiểm tra cho từng sinh viên.

```
26 // Determine each student's average score.
27 for (int student = 1; student <= numStudents; student++)
28 {
29     total = 0; // Initialize the accumulator.
30     for (int test = 1; test <= numTests; test++)
31     {
32         double score;
33         cout << "Enter score " << test << " for ";
34         cout << "student " << student << ": ";
35         cin >> score;
36         total += score;
37     }
38     average = total / numTests;
39     cout << "The average score for student " << student;
40     cout << " is " << average << ".\n\n";
41 }
```

Inner Loop

Outer Loop

Thực hành: Vòng lặp lồng

// Màn hình vuông:
// Yêu cầu người dùng nhập một số nguyên dương không lớn hơn 15. Sau đó, hiển thị một hình vuông
// trên màn hình bằng ký tự 'X' với chiều dài cạnh là số người dùng nhập.

```
#include <iostream>
using namespace std;
int main()
{
    int n;
    do{
        cout << "n (trong khoang 1-15): ";
        cin >> n;
    }
    while (n < 1 || n > 15);

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            cout << 'X';
            cout << endl ;
        }

        return 0;
    }
}
```

```
n (trong khoang 1-15): 17
n (trong khoang 1-15): 0
n (trong khoang 1-15): 6
XXXXXX
XXXXXX
XXXXXX
XXXXXX
XXXXXX
XXXXXX
```

Chạy xong vòng con tương đương với in hết một dòng, nên xuống dòng

Thực hành: Vòng lặp lồng

// vẽ tam giác vuông xuôi và ngược bằng dấu + với chiều dài cạnh nhập từ bàn phím

```
int n;
cout << "n: ";
cin >> n;

//vẽ tam giác vuông xuôi:
for (int i = 1; i <= n; i++)
{
    for (int j = 1; j <= i; j++)
        cout << "+";
    cout << endl;
}
```

```
n: 4
+
++
+++
++++
++++
+++
++
+
++++
+++
++
+
```

```
//vẽ tam giác vuông ngược cách 1:
//đánh số dòng tăng dần thì dòng i có n+1-i dấu cộng
for (int i = 1; i <= n; i++)
{
    for (int j = 1; j <= n + 1 - i; j++)
    {
        cout << "+";
    }
    cout << endl;
}
```

```
//vẽ tam giác vuông ngược cách 2:
//đánh số dòng giảm dần thì dòng i có đúng i dấu cộng
for (int i = n; i >= 1; i--)
{
    for (int j = 1; j <= i; j++)
        cout << "+";
    cout << endl;
}
```

Mở rộng: Nhập xuất file

- ❑ Các đơn vị lưu trữ dữ liệu
- ❑ File và dòng

Các mức độ lưu trữ dữ liệu

- Từ nhỏ nhất đến lớn nhất:
 - **Bit**
 - **Byte**: gồm 8 bit, còn gọi là 1 ký tự
 - **Trường** (field): một nhóm ký tự có nghĩa (vd trường tên, tuổi, ..)
 - **Bản ghi** (record): một nhóm các trường liên quan
 - Vd: tên, mã, ngày sinh, ..., tạo thành bản ghi sinh viên
 - **File**: gồm các bản ghi cùng kiểu. VD file sinhvien.txt
 - **Cơ sở dữ liệu**: gồm các file liên quan
 - CSDL phục vụ đào tạo có file sinhvien.txt, giaovien.txt, mon.txt, ..

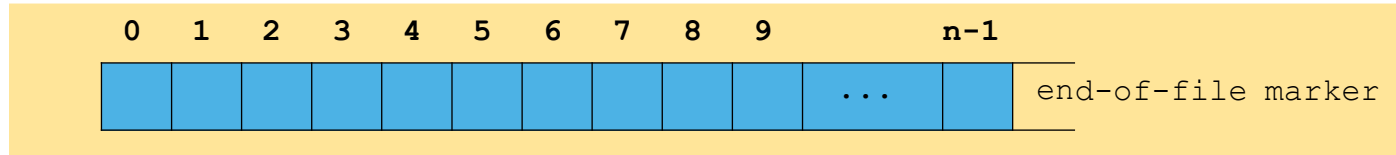
File và dòng

- C++ coi đọc ghi dữ liệu như việc nối ống nước tới nguồn nước/bể chứa. cin, cout là ống nhập chuẩn (từ bàn phím) và xuất chuẩn (ra màn hình).
- Đọc/ghi file chính là tạo ra ống, nối tới file, rồi dùng ống hết cin cout
- Tạo ra ống = khai báo các biến thuộc kiểu ống
 - Ống để đọc từ file = biến kiểu ifstream
 - Ống để ghi ra file = biến kiểu ofstream
 - Ống hai chiều, có thể cả đọc cả ghi = biến kiểu fstream
 - Các kiểu trên đều có trong thư viện <fstream>
- Có thể nối ống tới file ngay khi khai báo ống:

```
#include <fstream>
int main(){
    ifstream fin("class.txt"); // tạo ống fin để đọc từ file class.txt
    ofstream out("book.txt"); // ống out để ghi ra file books.txt
    fstream f("student.txt"); // ống f để vừa đọc vừa ghi ra student.txt
    if (!fin) cout << "Loi: Khong the mo file\n"; // nếu fin không nối được
    else { // nếu fin nối được thì từ đây sẽ làm việc với file
        //...
    }
}
```

Đọc và ghi file

- File giống như một dòng nước, cuối dòng là ký tự đặc biệt kết file (EOF)



- Một ống đọc lấy dữ liệu từ file hết cách cin lấy từ bàn phím
 - ❑ `fin >> n;` // đọc dữ liệu đầu tiên khác trắng
 - ❑ `fin >> int1 >> char2 >> bool3;` // đọc kiểu lợp ngói
 - ❑ `getline(fin, s)` // đọc từ đầu đến khi gặp enter
 - ❑ `fin.ignore(); fin.ignore(100, '\n');` // nhảy qua
 - ❑ `fin.get(c)` // đọc 1 ký tự
 - ❑ `if (fin >> a) {...}` // kiểm tra liệu việc đọc ổn thoả

Đọc và ghi file

- Một ống ghi in ra file hết cách cout in ra màn hình:
 - ❑ `out << n;` // in ra một số
 - ❑ `out << int1 << “ ” << string1 << endl;` // in kiểu lợp ngói
 - ❑ `out << setprecision(2) << fixed << float1;` // định dạng
 - ❑ `out << setw (8) << int1 << ‘\n’;` // giống hàng
- Một ống fstream có thể đọc lẫn ghi, miễn kết hợp đúng toán tử
 - ❑ `f >> n;` // toán tử >> để đọc
 - ❑ `f << “Hello” << endl;` // toán tử << để ghi
- Khi làm việc xong với file, cần tháo ống khỏi file bằng hàm close:
 - ❑ `fin.close();`
 - ❑ `out.close();`
 - ❑ `f.close();`

Tổng kết quy trình đọc và ghi file

- 1) Thêm thư viện <fstream>
- 2) Khai báo biến kiểu phù hợp và nối biến với tên file muốn xử lý
ostream out ("data.txt");
tương đương
ostream out;
out.open("data.txt");
- 3) Kiểm tra có nối thành công
if (!out) // ... không nối được thì thông báo lỗi mở file
else // ... nối được thì đặt đoạn mã xử lý file ở đây
- 4) Xử lý file (đọc, ghi, cả đọc cả ghi)
in >> n;
out << n << endl;
- 5) Đóng kết nối
in.close(); out.close();

Ví dụ: Đọc file

// File random.txt chứa các số ngẫu nhiên. Đọc file này, rồi hiển thị:
// Số lượng, tổng và giá trị trung bình của các số trong file

```
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
    ifstream fin("random.txt");

    if (!fin)
        cout << "Loi: khong doc duoc file \n";
    else {
        double n, tong = 0;
        int dem = 0;
        while (fin >> n) //còn đọc được thì còn làm tiếp;
        {
            dem ++;
            tong += n;
        }
        cout << "Tong: " << tong << endl
              << "So luong: " << dem << endl;
        if (dem)
            cout << "Trung binh: " << tong/dem << endl;
        else
            cout << "Khong tinh duoc trung binh \n";
    }
    fin.close();

    return 0;
}
```

Tạo ống fin và nối nó với
nguồn random.txt

Nếu nối ống không được, fin sẽ
là false, ngược lại là true

fin >> n đọc dữ liệu file vào biến n, nó cũng trả
về true nếu đọc thành công, và false nếu không
đọc được (do hết file, dữ liệu sai kiểu, ...)

Dùng xong thì gỡ ống khỏi nguồn. Ống fin lúc này
vẫn tồn tại, chỉ đang không nối tới nguồn nào

Ví dụ: Ghi file

// Nhập vào một số n, sau đó ghi số đó vào dòng đầu file number.txt. Sau đó nhập từ bàn phím
// lần lượt nhập n số và ghi chúng vào dòng thứ 2 trong file number.txt, cách nhau bởi dấu cách.

```
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
    int n;
    cout << "n: ";
    cin >> n;

    ofstream fout ("number.txt");
    if (!fout)
        cout << "Loi, khong ghi duoc file" << endl;
    else {
        fout << n << endl;
        for (int i = 0; i < n; i++){
            double so;
            cout << i << " : ";
            cin >> so;
            fout << so << " ";
        }
    }
    fout.close();
    return 0;
}
```

Tạo ống fout và nối nó đồ
vào number.txt

Nếu nối ống không được, fin sẽ
là false, ngược lại là true

fout << ghi dữ liệu ra file

Dùng xong thì gỡ ống khỏi nguồn. Ống fout lúc này
vẫn tồn tại, chỉ đang không nối tới nguồn nào

Cả đọc lần ghi file

Vẫn đọc file,
nhưng nhập tên
file từ bàn
phím, và in
kết quả ra
file thay vì
ra màn hình

```
int main()
{
    string finName, foutName;    // tên file sẽ đọc, tên file sẽ ghi
    cout << "Ten file de doc: ";
    cin >> finName;
    cout << "Ten file de ghi:";
    cin >> foutName;
    ifstream fin(finName.c_str()); // ống đọc file, cần chuyển tên file thành dạng c-string

    if (!fin)
        cout << "Loi, khong doc duoc file" << endl;
    else {
        double n, tong = 0;
        int dem = 0;
        while (fin >> n)           //còn đọc được thì còn làm tiếp;
        {
            dem ++;
            tong += n;
        }
        ofstream fout(foutName.c_str()); //khai báo ống để ghi file
        if (!fout)
            cout << "Loi: khong ghi duoc file \n" ;
        else{
            fout << "Tong cac so: " << tong << endl
            << "So cac so: " << dem << endl;
            if (dem)
                fout << "Trung binh cac so: " << tong/dem << endl;
            else
                fout << "Khong tinh duoc trung binh \n";
        }
        fout.close();
    }
    fin.close();
    return 0;
}
```


Thực hành: Đọc ghi file

```
// Vẫn đọc từ file random.txt, tính toán các thông tin sau về file:  
// a.   Số số số chẵn  
// b.   Tổng các số lẻ  
// c.   Giá trị trung bình của các số chia hết 3  
// rồi ghi các kết quả ra file result.txt
```

```
10  #include <iostream>  
11  #include <fstream>  
12  using namespace std;  
13  
14  int main()  
15  {  
16      ifstream fin("random.txt");  
17      if (!fin)  
18          cout << "Loi: khong doc duoc file \n";  
19      else {  
20          int n, tongLe = 0, tongChia3 = 0;  
21          int demChan = 0, demChia3 = 0;  
22          while (fin >> n) //còn đọc được thì còn làm tiếp;  
23          {  
24              if (n % 2 == 0)  
25                  demChan ++;  
26              else  
27                  tongLe += n;  
28              if (n % 3 == 0) {  
29                  demChia3++;  
30                  tongChia3 += n;  
31              }  
32          }  
}
```

Thực hành: Đọc ghi file

```
33     ofstream fout ("result.txt");
34     if (!fout)
35         cout << "Loi: Khong ghi duoc file \n";
36     else{
37         fout << "So so chan: " << demChan << endl
38             << "Tong so le: " << tongLe << endl;
39         if (demChia3)
40             fout << "Trung binh cac so chia het 3: " << double(tongChia3) / demChia3 << endl;
41         else
42             fout << "Khong co trung binh cac so chia het 3 \n";
43     }
44     fout.close();
45 }
46 fin.close();
47 return 0;
48 }
```

Cả đọc lần ghi file

vẫn xử lý số,
nhưng nhập tên
file đọc, ghi
từ bàn phím

```
string finName, foutName;    // tên file sẽ đọc, tên file sẽ ghi
cout << "Ten file de doc: ";
cin >> finName;
cout << "Ten file de ghi:";
cin >> foutName;
ifstream fin(finName.c_str()); // ồng đọc file, cần chuyển tên file thành dạng c-string

if (!fin)
    cout << "Loi, khong doc duoc file" << endl;
else {
    int n, tongLe = 0, tongChia3 = 0;
    int demChan = 0, demChia3 = 0;
    while (fin >> n) //còn đọc được thì còn làm tiếp;
    {
        if (n % 2 == 0)
            demChan ++;
        else
            tongLe += n;
        if (n % 3 == 0) {
            demChia3++;
            tongChia3 += n;
        }
    }
    ofstream fout(foutName.c_str()); //khai báo ồng để ghi file
    if (!fout)
        cout << "Loi: Khong ghi duoc file \n";
    else{
        fout << "So so chan: " << demChan << endl
            << "Tong so le: " << tongLe << endl;
        if (demChia3)
            fout << "Trung binh cac so chia het 3: " << double(tongChia3) / demChia3 << endl;
        else
            fout << "Khong co trung binh cac so chia het 3 \n";
    }
    fout.close();
}
fin.close();
return 0;
```

Tổng kết về vòng lặp

- Phép toán hữu ích cho vòng lặp: phép gán kết hợp, phép tăng, giảm
- Vòng lặp while, for, do/while
- Lệnh break và continue
- Lựa chọn vòng lặp phù hợp bài toán: số bước lặp xác định, xác minh dữ liệu, hiển thị menu, xử lý dữ liệu kết bởi canh, biến tích lũy
- Mở rộng: vòng lặp lồng

Tổng kết về file

- Các loại “ống” để đọc ghi file
- Quy trình đọc ghi file:
 - tạo ống
 - nối ống tới file
 - nếu nối không được, báo lỗi
 - nếu nối được thì dùng ống để đọc file
 - dùng xong thì tháo ống khỏi file

Trò chơi đoán số (tiếp)

Máy tính nghĩ ra một số ngẫu nhiên trong khoảng từ 1 đến 1000, nó giữ bí mật và mời bạn đoán. Mỗi lần đoán, nó sẽ cho bạn thắng nếu bạn trả lời đúng số cần đoán. Nếu không đúng, bạn được biết giá trị vừa đoán là quá cao hay quá thấp so với số cần đoán. Sau 10 đoán mà vẫn không đoán trúng thì bạn sẽ thua.

Hello! What is your name?

Mai

Play with me, Mai! I am thinking of a number between 1 and 1000.

Take a guess.

10

Your guess is too high.

Take a guess.

2

Your guess is too low.

Take a guess.

4

Well done, Mai! You guessed my number in 3 guesses!

Trò chơi đoán số - Tuần 4-5

- Tuần này ta có thể tích hợp thêm những gì mới học:
 - Xác thực đầu vào: nếu nhập sai (không đúng kiểu, ngoài khoảng 1, 1000) thì lại mời nhập
 - Được đoán nhiều nhất 10 lần thay vì chỉ 1 lần:
 - Nếu đoán sai, thông báo đoán quá cao hay quá thấp. Nếu chưa đoán đủ 10 lần thì mời đoán tiếp, nếu đủ thì thông báo đã hết lượt đoán.
 - Nếu đoán đúng thì thông báo chúc mừng đã thắng trò chơi sau bao nhiêu lượt đoán.
 - Khi đoán đúng lần khi hết lượt đoán, không kết thúc trò chơi mà hỏi muốn chơi lần nữa hay dừng, tùy câu trả lời mà sẽ chơi lại (máy tính nghĩ ra 1 số ngẫu nhiên mới, trò chơi bắt đầu từ đầu) hay kết thúc chương trình.

Trò chơi đoán số - Tuần 4-5

- ❑ Ghi lại kết quả của người chơi vào một file : tên, số lần, kết quả từng lần.
- ❑ Nhập tên một người chơi cụ thể, đọc file và in ra kết quả các lần chơi của người chơi đó
- ❑ Đọc file, in ra tên và kết quả của người chơi vô địch. Người chơi vô địch là người có tỷ lệ số lần thắng trên tổng số lần chơi là cao nhất
- ❑ Tự thiết kế cấu trúc bản ghi của file để tiện cài đặt các chức năng trên